Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Security and ‘Auth’

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

- Reasons for trust and locations where trust is important
- Methods for giving, assessing and managing trust
- **Auth**entication
- **Auth**orisation
- Impersonation
- Certification and the use of trusted parties

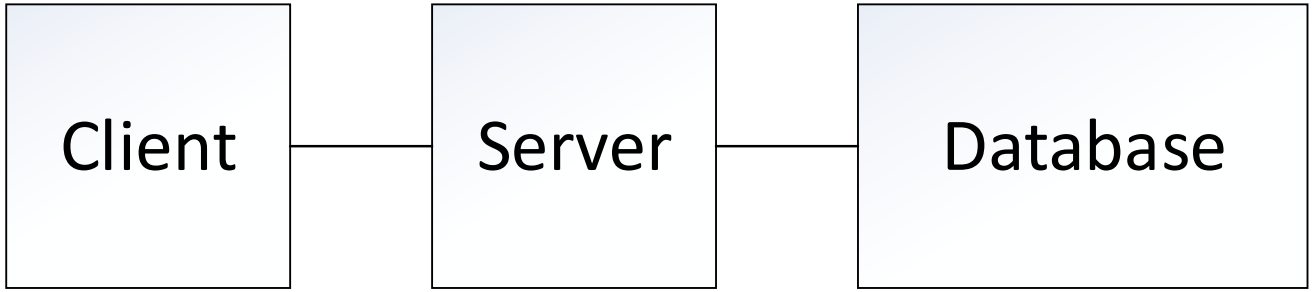
UNIVERSITY of HULL

Protecting Assets

- In the last session we looked at identifying threats
- Once you've identified what the threats are, it's a good idea to actually protect your assets
- Very, very frequently this is only possible if you can introduce some form of trust
- Between two interconnected systems trust must go two ways
- Between many interconnected systems, each device must trust everything it is connected to, everything those devices are connected to… and so on…

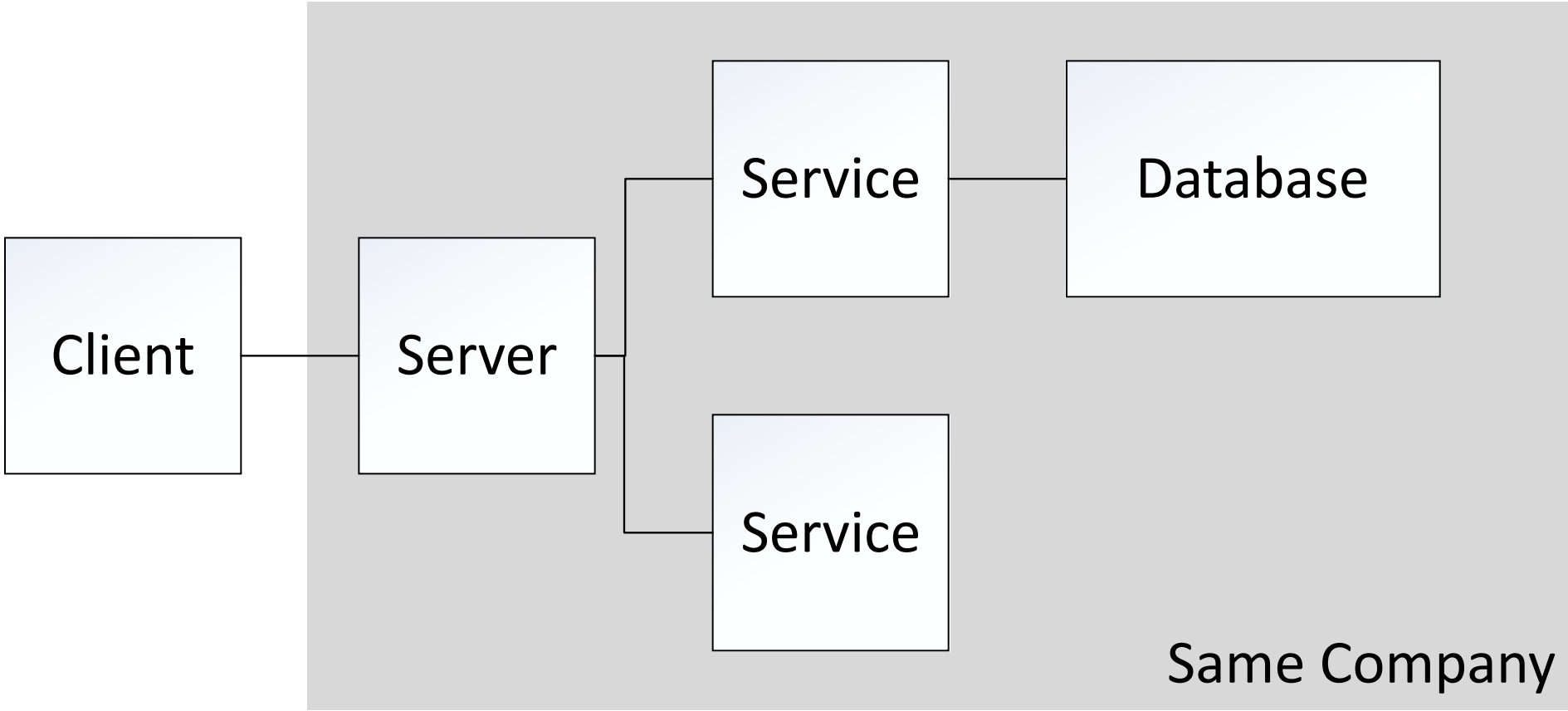
UNIVERSITY of HULL

Where is Trust Required?



UNIVERSITY of HULL

Where is Trust Required Here?



UNIVERSITY of HULL

Trust

- It's usually good to establish trust between **ALL** distributed components – no matter where they are located.

- This is true over the internet but is still true even for components owned by the same person/company/organisation, on the same subnet, or even on the same machine.

Why?

- A system is only as strong as its weakest link.
- Distribution means connectivity, connectivity means relying on data transfer
- Data transfer SHOULD immediately make you worried about security as you are inherently relying on all the hardware and software in between your endpoints

UNIVERSITY of HULL

Authentication

Is the person/service trying to log in authentic?

Two questions:

- *Who are you?*
- *Is it really you?*

Common answers:

- Who are you? – Username, Email address, ID
- Is it really you? – Password, certificate, token, biometric data, SMS code (two-factor authentication)

UNIVERSITY of HULL

Passwords

- Passwords are usually **hashed** to avoid saving the plaintext password itself to the password database.

- Hashing is a repeatable one-way process *that we will look at in detail in the next session*.

- The password is encrypted by the sender and sent to the other endpoint. There it is decrypted and then hashed to find out if the hash is the same as the one stored for the user.

- If yes, user is **authenticated**. If no, user is denied.
  Either way the plaintext password must be destroyed and not stored.

We will also look at encryption over the next few sessions.

UNIVERSITY of HULL

Passwords

- Does this mean that the password is safe?

Yes and no…

- Usually it is reasonable to assume that, whilst in plaintext, the password is safe on the client/server machines. This may be a foolish assumption so we should (at least):
  - minimise the amount of time the plaintext password is available, and
  - maximise protection from viruses or other malicious software

The hash may already be cracked…

Cracked Passwords

Most companies use one of a few open-source hashing algorithms.

- This is a good thing usually – it ensures your algorithm has been checked by lots of people who are actively interested in its security!
- But also bad… once someone finds a vulnerability or cracks a password it can cause many systems to be insecure at once.

Hash crackers spend computer time hashing every possible permutation of character to create reverse-lookup tables. This is damaging for short passwords:

- “abcd”, “8ns2!”

Dictionary attacks spend computer time hashing obvious passwords, words and phrases to create reverse-lookup tables.

This is particularly damaging for lazy passwords:

- “password”, “p@ssword1”, “12345678910”, “HelloWorld”

UNIVERSITY of HULL

Encryption

Even if we trust the endpoints we can never trust Internet connections

- Typically we need some way to ensure our passwords/data/messages are secured during transit along untrusted connections

We have two options for this:

- **Symmetric Encryption**: both endpoints use the same key to encrypt and decrypt messages
- **Asymmetric Encryption**: two different keys. Any endpoint uses a **common public key** to encrypt messages, the other endpoint uses a **private key** to decrypt

We'll go into this is lots more detail over the coming sessions

UNIVERSITY of HULL

Problem…

A web developer wants to send a message to a server securely

- This means sending an encrypted message
- Encrypted messages need to be **encrypted with a KEY** before sending and **decrypted with a KEY** at the other end.

But unless the server already knows the client, it doesn't know what key to use to decrypt the message.

The client could send a KEY with the message… but this could be intercepted.
- Someone else could get the key, then decrypt the subsequent message using the key
- This defeats the purpose of encryption!

UNIVERSITY of HULL

Why do we need Certificates?

An asymmetric **public KEY** can be sent to anyone to perform encryption.
It can't be used for decryption. Only the **private KEY** can be used for decryption.

1. The server can send its **public KEY** to the client.
2. The client can then use the **public KEY** to encrypt a message and send it to the server
3. The server receives an encrypted message. Decrypts it using its **private KEY** and gets the original plaintext

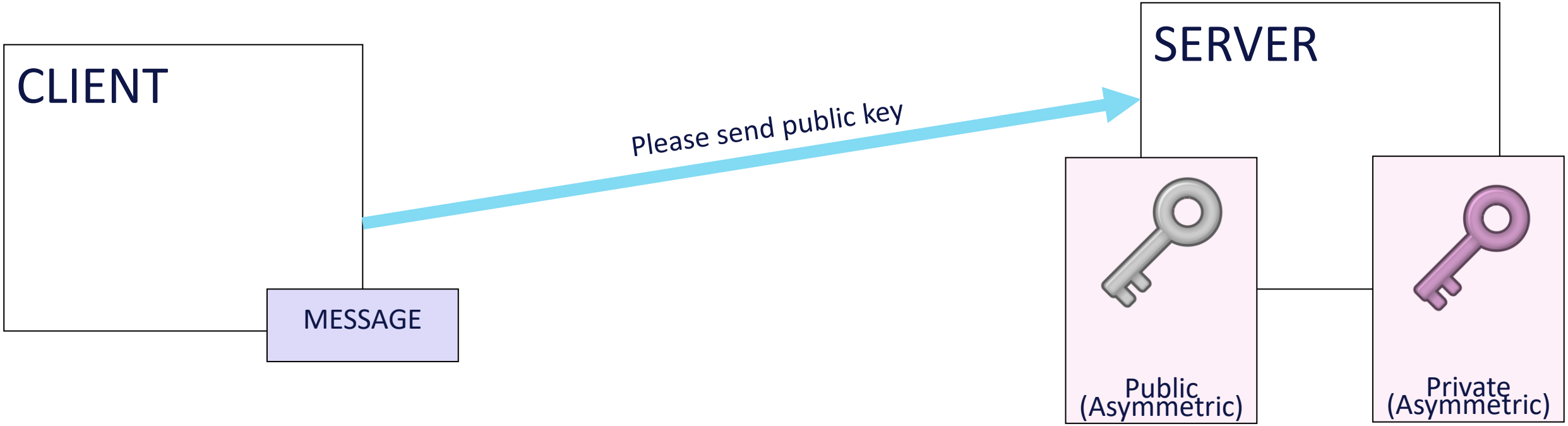
UNIVERSITY of HULL

Why do we need Certificates?

Client wants to send an encrypted message to the server

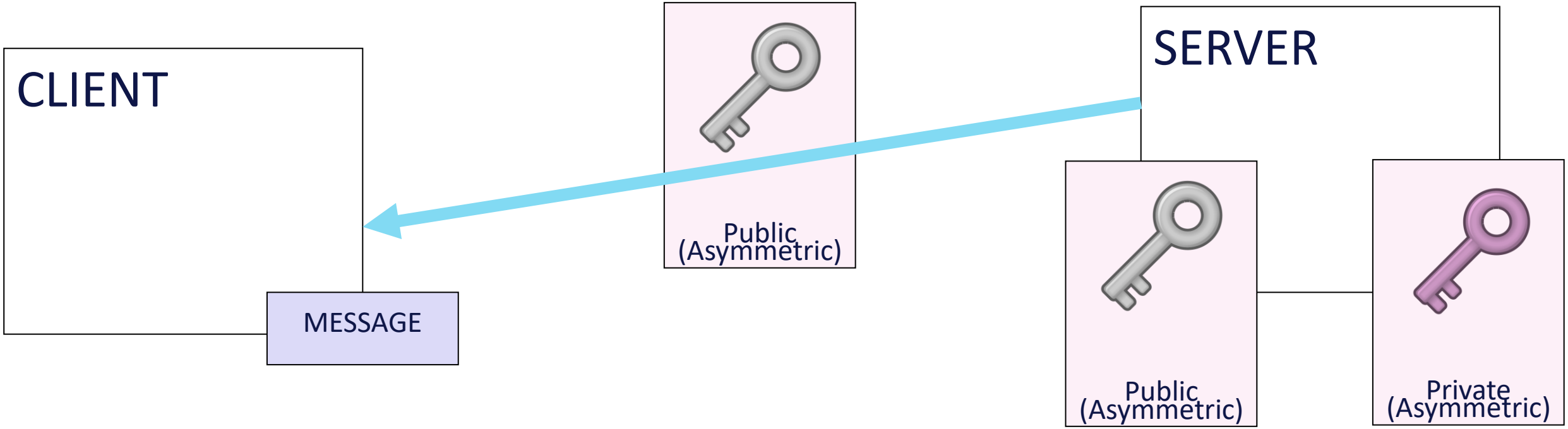
Why do we need Certificates?

Client asks server for public key…



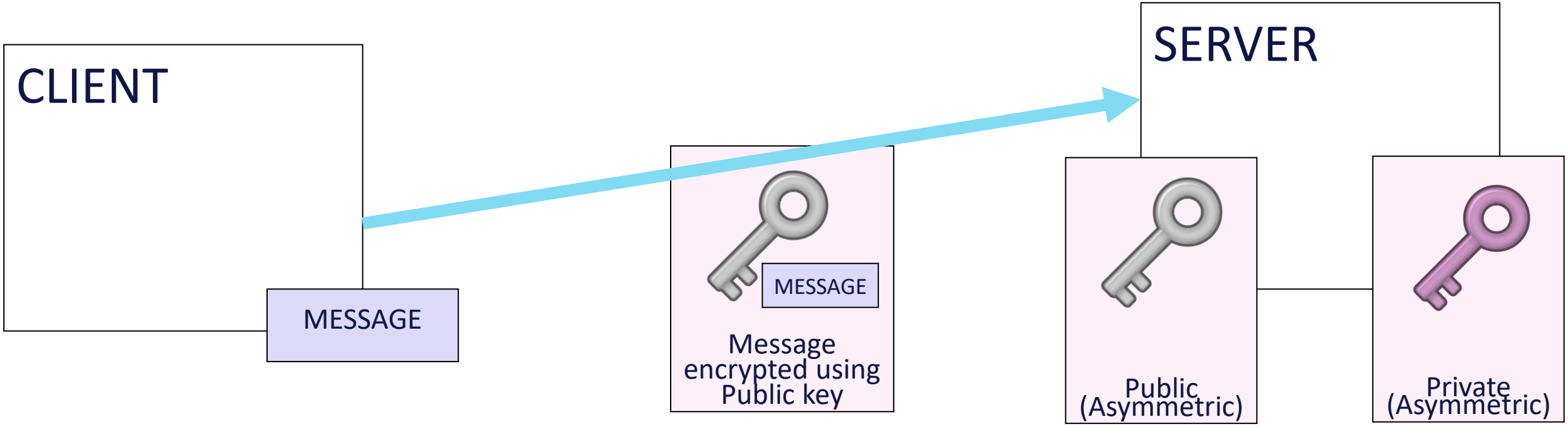
Why do we need Certificates?

Server sends public key. A public key can ONLY be used to encrypt, not decrypt.



Why do we need Certificates?

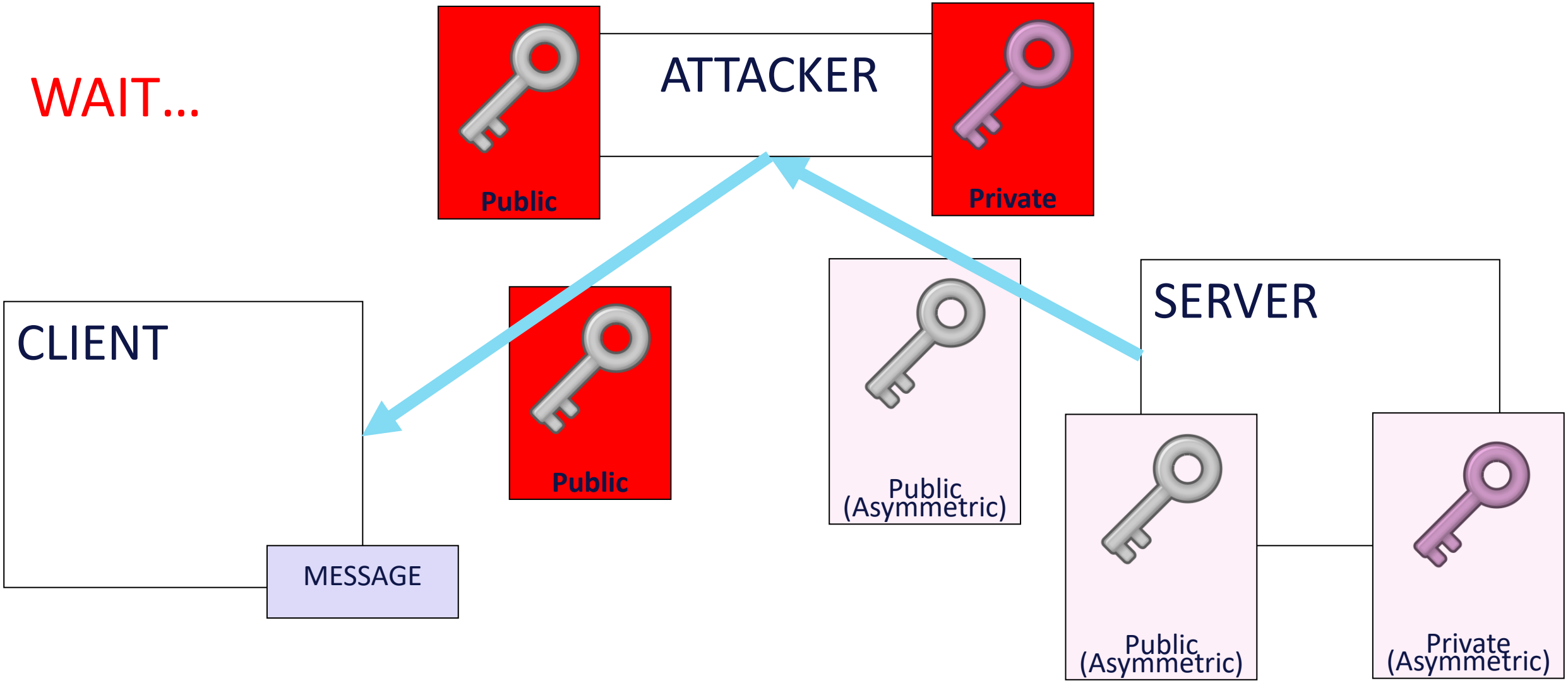
Client encrypts the message with the public key and sends the encrypted message to the server



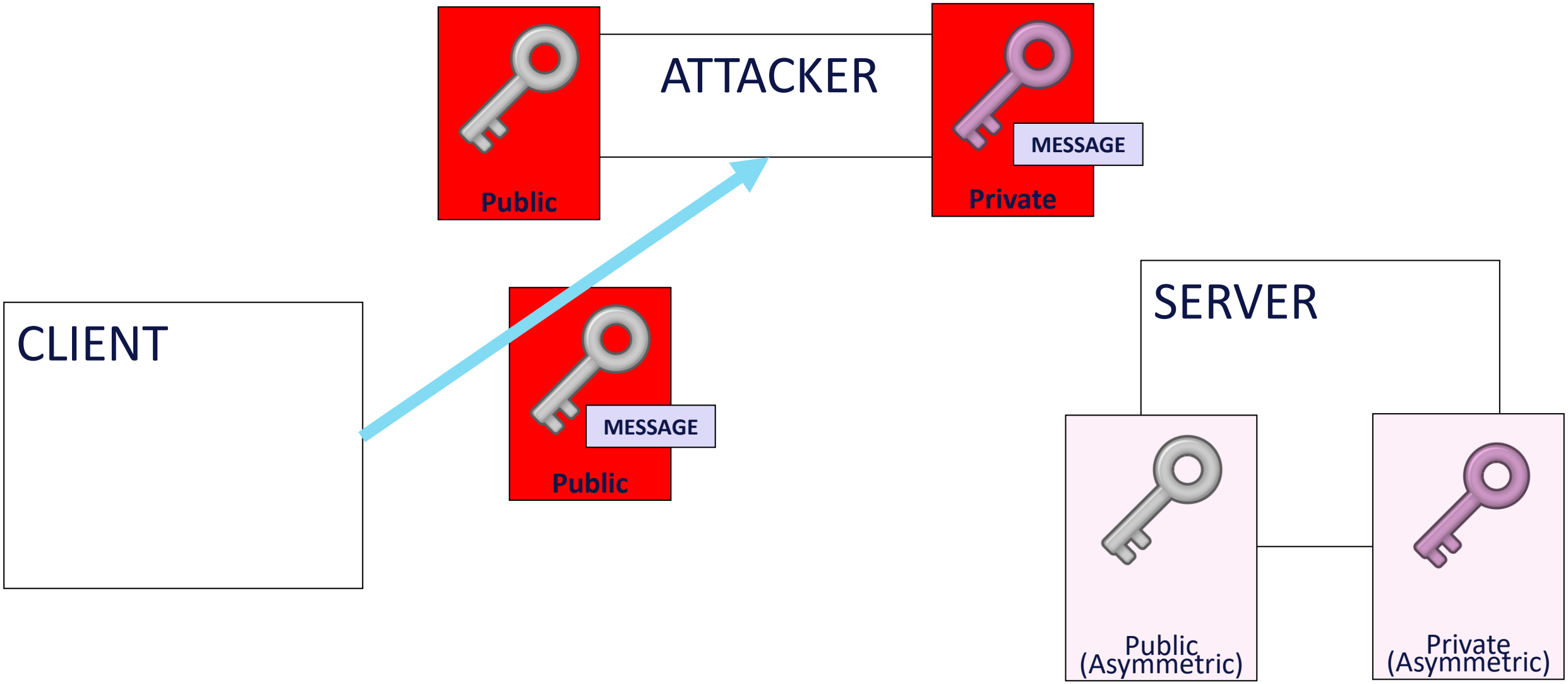
Why do we need Certificates?

Server decrypts the message using the private key. It now has the original message!

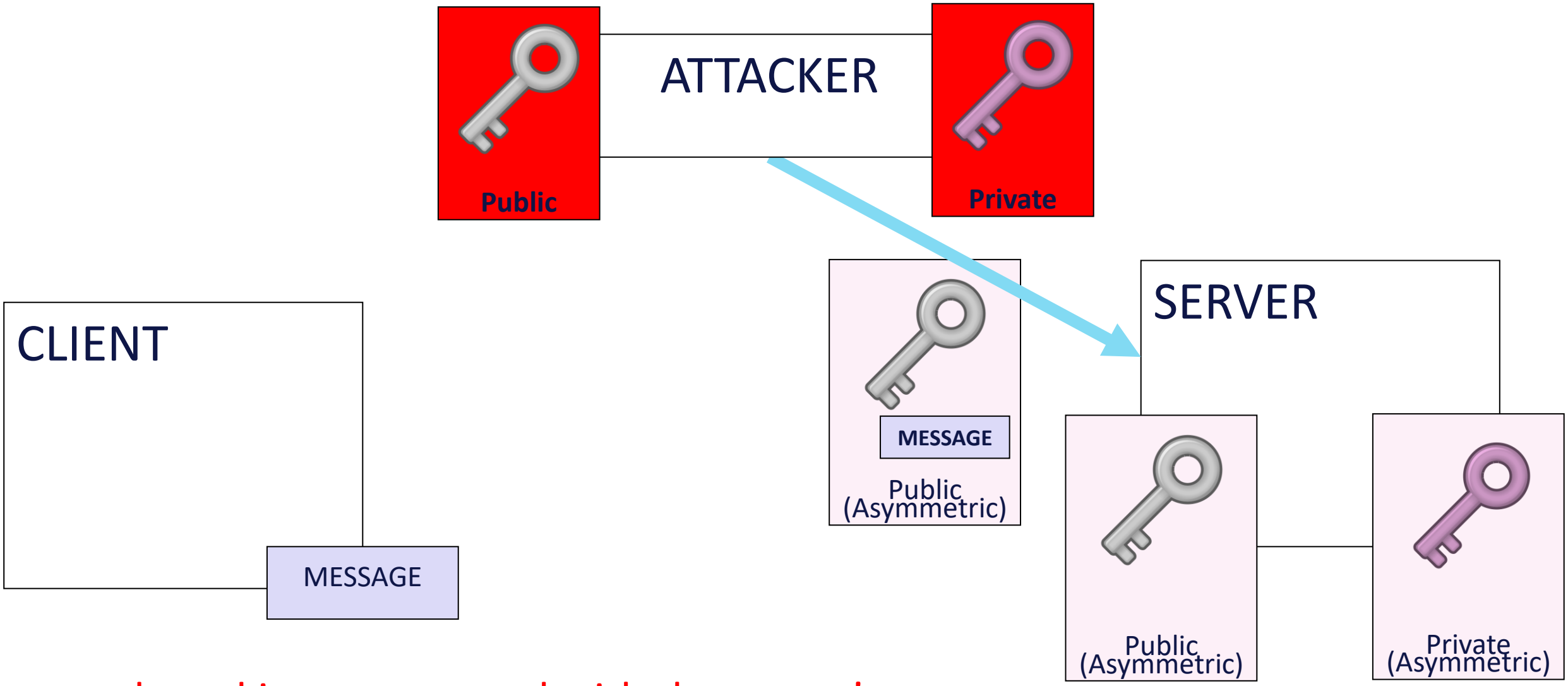
WAIT…



An attacker replaced the public key



Attacker can now use its private key to get the plaintext!



… and send it on encrypted with the server's public key as if nothing happened!

Why do we need Certificates?

We need to know that the public key from the server

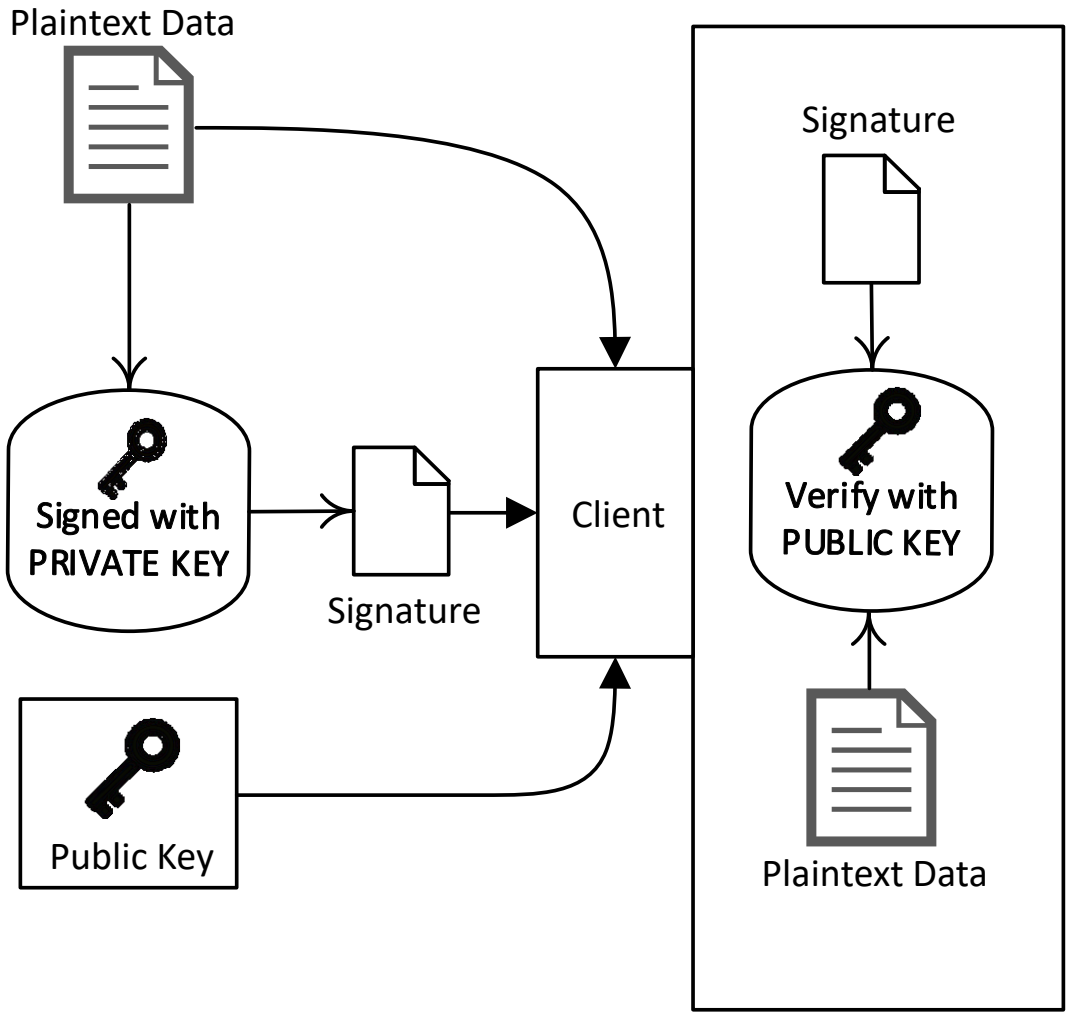
1. Definitely came from the server
2. Was not intercepted

For this we need to use another very useful feature of asymmetric cryptography:

- Data can be **SIGNED** with the private key.
- The signed data can be verified using the corresponding public key and the plaintext data

UNIVERSITY of HULL

What is Signing?



Note that usually we sign a hash of the data not the data itself – for speed (most signing algorithms do this for us)

This example is basic – it shows the concept of signing data.

We want to send some data…

- To prove that it came from the server it needs to be signed with a private key
- But the client doesn't have the corresponding public key yet to verify the signature!

We use a middleman, known to and trusted by both the server and the client (called a certificate authority) to verify the public key actually came from the server.

UNIVERSITY of HULL

How do we manage this in the real world?

- Public Key Infrastructure (PKI)…

"a set of hardware, software, people, policies, and procedures needed to create, manage, distribute, use, store, and revoke digital certificates based on asymmetric cryptography" (RFC 4949)

- Authorities / directories in PKI
  - **Certificate Authority**: issues and verifies certificates
  - **Registration Authority**: verifies identity of entities who ask for certificates
  - **Central directory**: stores certificates and revocation lists

UNIVERSITY of HULL

Certificates

- Certification Authority (**CA**)

<u>A known, trusted entity.</u> In HTTPS, **browsers have a list of trusted CA's & store the public keys of the CA's**.

– Common certificate example:

- Server sends its public key to trusted **CA**
- **CA** takes public key and signs it using the private key of **CA** and packages this into a certificate
- Certificate is sent to and stored in Server
- Client requests public key from server
- Server sends the Client the certificate, containing its public key and the signed version of the public key

– Pre-shared public key of known **CA** can be used with Server's public key to verify the certificate was issued by the **CA** and that the public key is the one originally signed by the **CA**

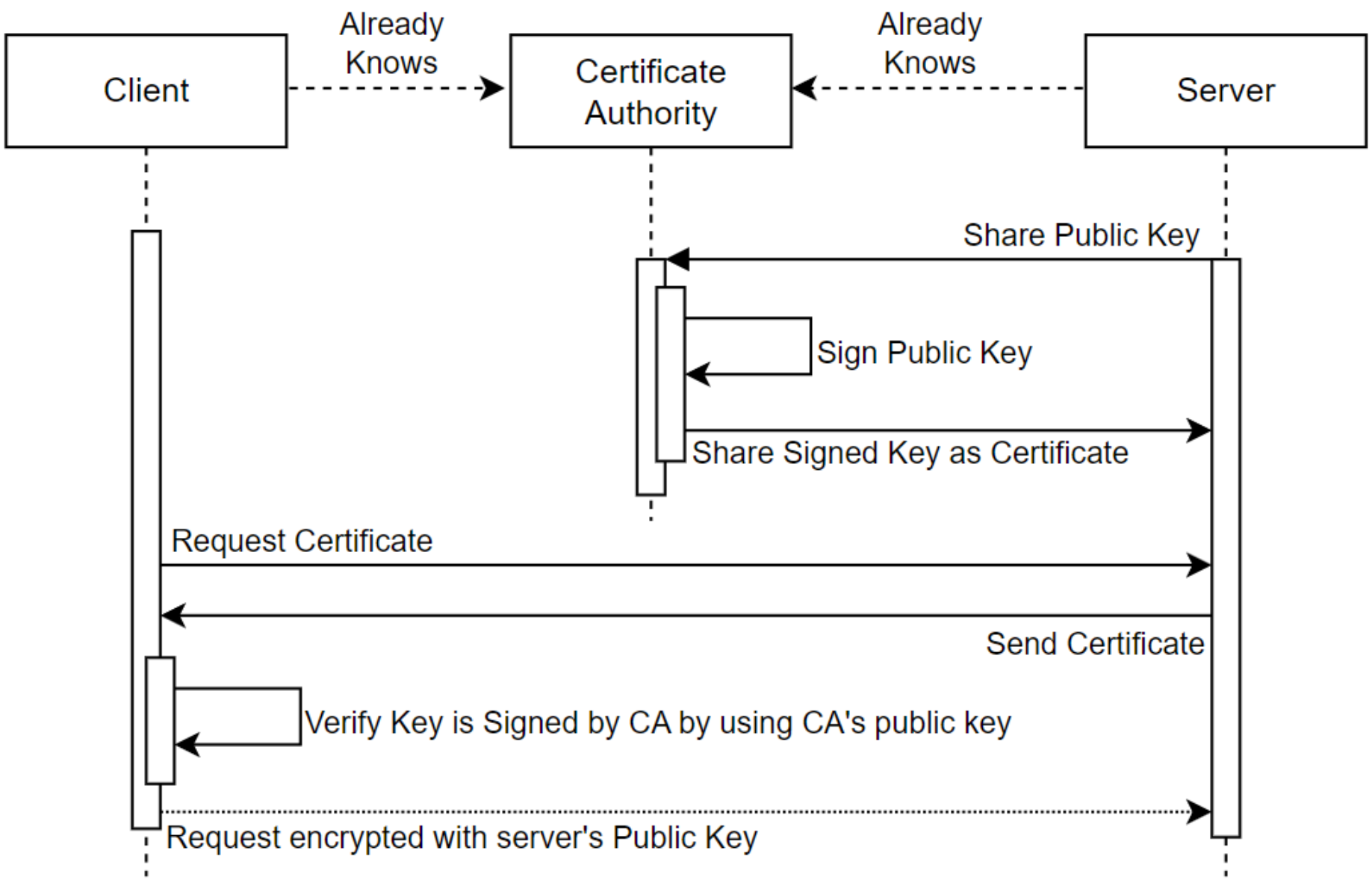
- X.509 Digital Certificates

– Compatible protocols include TLS, SSH, LDAP

– Standard format for certificates

UNIVERSITY of HULL

Certificates



Authorisation

Is the person/service trying to perform a function authorised to do so?

- Asks the question:
  - *Are you **allowed** to do this?*

- May be answered by:
  - Access control lists
  - User roles
  - Capabilities
  - Policies

- One or more of these methods may be used at the same time

UNIVERSITY of HULL

Authorisation Mechanisms

- Access Control Lists

  Access depends on user's identity appearing in a list

  e.g. the list of users
- User Roles

  Users may have one or more roles given to them.

  e.g. administrator, elevated access, normal user, basic user, guest, no access
- Capabilities

  Access depends on a capability

  e.g. verification phone calls in online banking
- Policies

  Group or system policies which give or revoke rights

  e.g. all admin users are allowed to do X

  all users are allowed to do Y

UNIVERSITY of HULL

Authorisation Control

Four types

- Mandatory Access Control

  e.g. security clearance

- Discretionary Access Control

  Decided by the data owner

- Role-Based Access Control

  Decided by membership in a user group

- Attribute-Based Access Control

  Decided by an attribute, e.g. over 18

Microsoft Web Authorisation

Microsoft calls it Role-based Security
(but we know by now that security is much more than just access control)

- Uses Identities, Claims and Roles:
  - A user's identity is made from Claims (sometimes called Scopes)
  - Claims identify 'things' about the user – e.g. their name
  - A Role is a specialised form of Claim - they designate levels of authorisation

- Identities don't explicitly grant any authorisation
  - An identity is approved authentication
- Claims (including Roles) can be given and revoked at any time

UNIVERSITY of HULL

Checking Identity in the Pipeline

We may want to check identity for authorisation/authentication in the pipeline.

This means that we have information about whether the request is allowed before we start handling the request.

Common work is done before the service endpoint methods are called
- e.g. populating an identity

UNIVERSITY of HULL

Impersonation

The server may have lots of (or no) access rights.

Clients shouldn't be given all the rights the server has (or restricted by how few the server has by default). Clients should be granted their own rights.

Impersonation is a security mechanism in client / server applications

- Allows the server to switch to the security context of the client
- If a client is allowed to run a protected behaviour, the server can now run it
- Without impersonating a client, the server can not run protected behaviours
- Good to avoid implementation of lengthy checks of access rights

Further Reading

- Tutorials on MSDN and other web resources
  - **Security, Authentication and Authorisation in ASP.NET**
    https://docs.microsoft.com/en-gb/aspnet/core/security/
  - ".NET Cryptography Model",
    https://msdn.microsoft.com/en-us/library/0ss79b2x%28v=vs.110%29.aspx
  - Impersonation in ASP.NET Core:
    https://learn.microsoft.com/en-us/dotnet/api/system.security.principal.windowsidentity.runimpersonated
  - Zero-Trust Guidance:
    https://learn.microsoft.com/en-us/security/zero-trust/

UNIVERSITY of HULL

Reflection

In this session we've looked at:

- Reasons for trust and locations where trust is important
- Methods for giving, assessing and managing trust
- Authentication
- Authorisation
- Impersonation
- Certification and the use of trusted parties

In the next two sessions we will be looking at the specifics of Hashing and Encryption

UNIVERSITY of HULL